Семейство языков ML

Основы и базовые конструкции

Standard ML OCaml F#

Данные и переменные, функции, управление потоком

Основные примеры: OCaml 5.4

История и представители семейства

1970-е

ML в проекте Edinburgh LCF

Группа Робина Милнера создаёт Meta Language для работы с доказательствами.

1990 / 1997

Standard ML

Формальное определение языка и пересмотренная редакция SML ’97.

1987 / 1996

Caml и Objective Caml

Публичный Caml, затем Objective Caml с объектной системой. Позднее: OCaml.

2002 / 2010

F# для .NET

Первый исследовательский релиз, затем поддерживаемый Microsoft F# 2.0.

Общие идеи ML

Статическая типизация

Типы проверяются до выполнения. Компилятор часто выводит их сам.

Данные с явной структурой

Кортежи, записи и варианты помогают описать допустимые состояния.

Функции как значения

Функцию можно передать в аргументе или получить как результат.

Строгие вычисления

Аргументы вычисляются до вызова. Изменяемое состояние допустимо.

Функциональная основа сочетается с императивными возможностями.

Спецификации и выбранные версии

Язык	Основа сравнения	Отличительная черта
Standard ML	SML ’97 Definition, Revised (1997)	Формальное определение и система модулей
OCaml	Reference Manual 5.4	ML-ядро, модули и объектная система
F#	Язык F# 9 Language Guide и RFC	Интеграция с .NET и его библиотеками

У семейства ML нет единой версии. Здесь зафиксирована база сравнения, а не список последних релизов.

Базовые типы данных

Назначение	Тип в OCaml	Примеры значений
Целые числа	int	42, -7
Вещественные числа	float	3.14, 2.0
Логические значения	bool	true, false
Символы и строки	char / string	'A' / "hello"
Единственное значение	unit	()

В Standard ML вещественный тип называется real. В F# доступны типы .NET, в том числе int, float и decimal. unit обозначает отсутствие значимых данных.

Составные и пользовательские типы

Кортеж

Несколько значений разных типов

```
let person = ("Ada", 20)
```

Запись

Именованные поля

```
type point = { x : float; y : float }
```

Вариант

Один из допустимых случаев

```
type shape = Circle of float | Point
```

Список

Однородная последовательность

```
let scores = [3; 4; 5]
```

Тип описывает форму данных. Обработка вариантов появится в match.

Операции над значениями

Операция	Пример OCaml	Результат
Целочисленная арифметика	7 / 2 7 mod 2	3 1
Вещественная арифметика	7.0 /. 2.0	3.5
Сравнение значений	3 = 3 3 < 5	true true
Логические операции	true && false	false
Строки и списки	"ML" ^ "!" 1 :: [2; 3]	"ML!" [1; 2; 3]

OCaml различает + и +., * и *., / и /. Оператор = сравнивает значения.
В Standard ML и F# набор операторов и правила перегрузки отличаются.

Преобразование и приведение типов

Числа преобразуются явно

Аннотация типа проверяет совместимость.
Она не выполняет числовое преобразование.

```
float_of_int 3       (* 3.0 *)
int_of_float 3.9     (* 3 *)
string_of_int 42     (* "42" *)
```

Разбор текста может не удаться

Вариант с option позволяет различить число и некорректную строку.

```
int_of_string_opt "42"
(* Some 42 *)
int_of_string_opt "abc"
(* None *)
```

(3 : float) вызовет ошибку типов. Для перехода int → float нужен float_of_int.
Приведение :> в OCaml применяется к совместимым подтипам, например объектам.

Связывания и области видимости

let связывает имя со значением

Локальное имя доступно в выражении после in. Область видимости лексическая.

Повторное имя затеняет прежнее

Новое связывание не изменяет старое. Снаружи остаётся исходное значение.

```
let x = 10

let y =
  let x = 3 in
  x + 1

(* y = 4, x = 10 *)
```

Standard ML: val и let ... in ... end. **F#: let и вложенность по отступам.**

Владение и совместное использование

Доступ после передачи

Передача значения в функцию не требует отдать исключительное владение им.

Общие изменяемые данные

a и b обозначают одну ячейку ref.
Изменение через b видно при чтении a.

```
let a = ref 0
let b = a

let set_to_seven r = r := 7
let () = set_to_seven b

let result = !a   (* 7 *)
```

Память освобождает сборщик мусора. let не делает содержимое ref неизменяемым.
В F# также есть let mutable x = 0 и присваивание x <- 7.

Функции: параметры и каррирование

Параметры передаются по значению

Аргументы вычисляются до вызова. Вызов записывается без обязательных скобок.

Частичное применение

add 2 создаёт функцию, которая прибавляет 2 к следующему аргументу.

```
let add x y = x + y
(* int -> int -> int *)

let add_two = add 2
let answer = add_two 5
(* 7 *)
```

int -> int -> int читается как int -> (int -> int).
Для кортежа: let add_pair (x, y) = x + y имеет тип int * int -> int.

Функции: результат и высший порядок

Результат тела становится ответом

Обычный return не нужен. Несколько результатов упаковывают в одно значение.

```
let divmod a b =
  (a / b, a mod b)

let pair = divmod 7 3
(* (2, 1), если b <> 0 *)
```

Функции принимают другие функции

Это позволяет отделить повторяемую схему вычисления от конкретного действия.

```
let twice f x = f (f x)
let r = twice (fun x -> x + 1) 5
(* 7 *)
```

unit: () служит аргументом или результатом, когда значимые данные не нужны.

Рекурсия и хвостовой вызов

Обычная рекурсия

```
let rec fact n =
  if n <= 1 then 1
  else n * fact (n - 1)
```

После возврата выполняется умножение.
Глубина стека растёт вместе с n.

Рекурсия с аккумулятором

```
let fact_tail n =
  let rec loop acc k =
    if k <= 1 then acc
    else loop (acc * k) (k - 1)
  in loop 1 n
```

Последняя операция: вызов loop.
OCaml может переиспользовать кадр стека.

Оба примера рассчитаны на n ≥ 0 и небольшие числа без переполнения int.
OCaml/F#: let rec. В Standard ML объявление fun уже допускает рекурсию.

Замыкания

Функция сохраняет окружение

Замыкание объединяет код функции и нужные ей значения из внешней области.

Новая функция при вызове

add_ten сохраняет n = 10 даже после завершения вызова make_adder.

```
let make_adder n =
  fun x -> x + n

let add_ten = make_adder 10
let result = add_ten 5
(* 15 *)
```

Если замыкание захватывает ref, оно видит текущее содержимое общей ячейки.

Условные выражения

if возвращает значение ветви

Условие имеет тип bool. В OCaml обе ветви должны иметь один тип результата.

Вычисляется выбранная ветвь

Условие можно использовать внутри другого выражения или функции.

```
let nonnegative x =
  if x >= 0 then x else 0

let sign =
  if 5 > 0 then "plus"
  else "not plus"
```

В OCaml if без else подразумевает else () и имеет результат unit.
В Standard ML ветвь else обязательна.

Сопоставление с образцом

match учитывает структуру данных

Выбирается первая подходящая ветвь.
Образец может извлечь часть значения.

Полное покрытие вариантов

Для списка есть два основных случая:
пустой [] и непустой head :: tail.

```
let first_or_zero xs =
  match xs with
  | [] -> 0
  | head :: _ -> head

let a = first_or_zero []
let b = first_or_zero [7; 8]
(* a = 0, b = 7 *)
```

OCaml и F#: match ... with. Standard ML: case ... of. _ игнорирует часть данных.

Циклы for и while

for: заданный диапазон

```
let total = ref 0
let () =
  for i = 1 to 3 do
    total := !total + i
  done
(* !total = 6 *)
```

while: пока условие истинно

```
let count = ref 0
let () =
  while !count < 3 do
    count := !count + 1
  done
(* !count = 3 *)
```

В OCaml границы for включены, результат циклов: unit. Есть направление downto.
В Standard ML встроен while, но нет for. В F# есть также for ... in ... do.

Итерация без ручного счётчика

List.iter

Выполнить действие для каждого элемента

```
List.iter print_int [1; 2; 3]
(* выводит 123, результат () *)
```

List.map

Построить новый список результатов

```
List.map (fun x -> x * 2) [1; 2; 3]
(* [2; 4; 6] *)
```

List.fold_left

Свернуть список в одно значение

```
List.fold_left ( + ) 0 [1; 2; 3]
(* 6 *)
```

Итераторы принимают функцию. map создаёт результат без изменения исходного списка.

Блоки и последовательность выражений

begin ... end группирует выражения

Это аналог скобок. Последовательность e1; e2 возвращает результат e2.

let ... in задаёт локальное имя

В примере имя x видно только внутри локального выражения. result равно 9.

```
let result =
  let x = 3 in
  begin
    print_endline "start";
    x * x
  end
```

Standard ML: let ... in ... end. В F# блоки обычно задаются отступами.
В OCaml ; разделяет вычисления, а ;; завершает фразу интерактивной среды.

Главные идеи и источники

Типы описывают данные

Вывод типов сокращает аннотации, сохраняя статическую проверку.

Функции строят вычисления

Каррирование, рекурсия и замыкания помогают переиспользовать код.

Управление потоком тоже опирается на выражения и их результаты.

Standard ML: история и определение
smlfamily.github.io/

OCaml: история языка
ocaml.org/history

OCaml 5.4: руководство
ocaml.org/manual/5.4/index.html

OCaml: учебные материалы
ocaml.org/docs

F#: спецификация и RFC
fsharp.org/specs/language-spec/

F# 9: руководство Microsoft
learn.microsoft.com · F# 9